Phase 0—StackMan

StackMan is a minimalistic programming language for which you will write an interpreter. You will use your interpreter to complete Assignment 5, which is due **February 19**.

The StackMan Language

Only the characters `0`, `+`, `<`, `>`, `?`, `_`, `~`, `(`, `)` are valid. Treat all others as comments—ignore them.

Program state is a stack of nonnegative integers. When the program starts, the stack is empty.

The `0` instruction pushes a zero on the top of the stack ("TOS"). We would say its behavior is:

```
... -> ... 0
```

That is, if it is executed against an empty stack ([]), the stack becomes $[0]$ afterward; if the stack is $[2, 3]$, then it will be $[2, 3, 0]$ afterward.

The + instruction adds one to TOS. We would say its behavior is:

```
... X -> ... X+1
```

Thus, if executed against $[2, 3]$, the stack would be changed to $[2, 4]$.

If + is executed against an empty stack, then—because TOS does not exist—we have an error condition ("stack underflow") and the interpreter exits with return code 1. Every instruction except `0` can underflow.

The _ ("drop") instruction drops TOS. Its behavior is:

```
... X -> ...
```

The ~ ("dup") instruction duplicates TOS; as before, an error condition occurs if it is executed against an empty stack. Its behavior is:

```
... X -> ... X X
```

The `?` ("test") instruction, if TOS is nonzero, decrements it and pushes a 1 on the stack; if TOS is zero, it pushes another zero; an error condition is raised if the stack is empty. That is, its behavior is:

```
... 0 -> ... 0   0
... X -> ... X-1 1   if X > 0
```

These instructions also “underflow” if run against an empty stack. In such conditions, the program should exit.

The `>` ("rot R") instruction pops TOS—call this value N—and, if N ≥ 2, rotates the top N values right like so:

```
...     Y X 2 -> ...     X Y
...   Z Y X 3 -> ...   X Z Y
... A Z Y X 4 -> ... X A Z Y
```

The `<` ("rot L") instruction is similar, but rotates left.

```
...     Y X 2 -> ...     X Y
...   Z Y X 3 -> ...   Y X Z
... A Z Y X 4 -> ... Z Y X A
```

If the stack is insufficient—e.g., if `>` is executed against $[0, 0, 3]$—then stack underflow occurs; a 3 is popped, but there are only two elements on the stack after this is done, so the instruction is illegal.

This language is concatenative. Therefore, αβ is the program that does α, then does β. For example, the program `0+0` consists of three instructions and its behavior is:

```
... -> ... 0 -> ... 1 -> ... 1 0
```

Another example, `0++<`, is called “swap”:

```
... X Y -> ... X Y 0 -> ... X Y 1 -> ... X Y 2 -> ... Y X
```

Note also that the empty string is a valid program—it does nothing; its behavior on the stack is the identity function.

Finally, the `(` and `)` characters represent a loop. The behavior of `(α)` is:

- if TOS is zero, do nothing.
- if TOS is nonzero, do α, then do `(α)`—that is, it functions like a `while`-loop using TOS as the test variable.

For example, the program `(?_0++<+0++<)_` has behavior:

```
... X Y -> X+Y
```

because, in a loop, it does:

```
... X Y -> ... X Y-1 1    // ?
        -> ... X Y-1      // _
        -> ... Y-1 X      // 0++< = "swap"
        -> ... Y-1 X+1    // +
        -> ... X+1 Y-1    // 0++<
```

until it reaches `... X+Y 0`, when it exits the loop, then drops the 0 with _.

If parentheses are not well-matched—that is, a `)` occurs before any opening `(`, or a `(` is found with no closing `)`—then this is an error condition.

Your Interpreter

You may assume that stack elements will never get larger than 65,535, though you are free to use a larger integer type.

The stack itself will never get larger than 65,536 elements—you should exit with a stack overflow if it does.

Your interpreter will run at the command line, taking one argument, which will be the name of a text file containing StackMan code. Starting with an empty stack, it runs that program. On successful execution, it should return:

- the number of instructions executed.
    - each loop test counts as an instruction; `)` is not an instruction.
- the contents of the stack—space separated, with TOS on right.

```
$ echo -n "0++++0+" > write_four_one.sm
$ ./stackman write_four_one.sm
# instructions: 7
final stack contents: 4 1

$ echo -n "0+++0++(?_0++<+0++<)_" > add_two_to_three.sm
$ ./stackman add_two_to_three.sm
# instructions: 33
final stack contents: 5
```

Use an exit code of 0 (that is, `return 0`) when the StackMan program successfully executes. However, there are five error conditions that you must be able to report, with their exit codes included:

- stack underflow (1)—an instruction demands more stack elements than exist.
- stack overflow (2)—the stack exceeds the buffer allocated to handle it.
- syntax error (3)—for example, unbalanced parentheses or invalid characters.

- unable to open file (4)—name corresponds to nonexistent or unopenable file.
- no filename given (5)—no filename supplied by user at command line.

Please include helpful error messages in these cases, e.g.:

```
$ echo "_" > underflow.sm
$ ./stackman underflow.sm
error: stack underflow
$ echo $?
1
```

You may assume that all code files will be smaller than 4 KB (4096 bytes.)